Implementación Paso a Paso del Algoritmo de Clustering Espectral

1. Algoritmo Spectral Clustering

El presente documento detalla la implementación completa de un algoritmo de *Spectral Clustering* aplicado a la segmentación de imágenes. El objetivo es tomar una imagen, analizar el color y la posición física de cada píxel, y agruparlos de forma automática para separar los objetos presentes del fondo de la imagen. A continuación, se explica y desarrolla cada fase del proceso.

1.1. Celda 1: Preparación del Entorno e Importación de Librerías

```python
# 1. Librerias matematicas y de grafos
import numpy as np
from scipy.spatial.distance import cdist
from scipy.sparse import csr_matrix, csgraph
from scipy.sparse.linalg import eigsh

# 2. Librerias de Machine Learning
from sklearn.cluster import SpectralClustering

# 3. Librerias de procesamiento de imagenes
from skimage import io, transform, color

# 4. Librerias visuales
import matplotlib.pyplot as plt
```

Listing 1: Importación de librerías esenciales

En esta primera fase se preparan las herramientas matemáticas e informáticas necesarias para el algoritmo. Se importan módulos esenciales como `numpy` y `scipy` para el cálculo de distancias (`cdist`), la construcción de la matriz Laplaciana (`csgraph.laplacian`) y la obtención de autovalores (`eigsh`). Asimismo, se incorpora la librería `scikit-learn` para el modelo de clustering y `skimage`/`matplotlib` para el procesamiento y visualización de imágenes.

1.2. Celda 2: Definición de la Función de Afinidad

```python
def calcular_mi_afinidad(dist_matrix, tipo='gaussiana', parametro=0.2):
    if tipo == 'gaussiana':
        # La clasica campana de Gauss
        return np.exp(- (dist_matrix ** 2) / (2.0 * parametro ** 2))
    elif tipo == 'lineal':
```

```
        afinidad = 1 - (dist_matrix / np.max(dist_matrix))
        return np.maximum(0, afinidad)
    elif tipo == 'umbral':
        return (dist_matrix < parametro).astype(float)
    elif tipo == 'cauchy':
        return 1.0 / (1.0 + (dist_matrix ** 2 / parametro ** 2))
    elif tipo == 'multicuadratica':
        return 1.0 / np.sqrt(dist_matrix ** 2 + parametro ** 2)
    elif tipo == 'paso_fino':
        afinidad = np.exp(- (dist_matrix ** 2) / (2.0 * parametro ** 2))
        afinidad[dist_matrix > parametro * 3] = 0
        return afinidad
    else:
        raise ValueError("Tipo de afinidad no reconocido")
```

Listing 2: Definición de funciones de afinidad

La función de afinidad establece cómo se traduce la distancia física o cromática entre dos píxeles en "afinidad" (es decir, cómo de similares o conectados están).

La imagen se trata como un grafo. Los píxeles son nodos y la afinidad es el grosor del cable que los une. Si dos píxeles son idénticos, su distancia es 0 y su afinidad máxima (1). Si son muy distintos, su afinidad tiende a 0.

Veamos los tipos de funciones de afinidad: La fórmula principal implementada es la función **Gaussiana** ($e^{-d^2/2\sigma^2}$), la cual penaliza fuertemente las distancias grandes para asegurar que sólo los píxeles verdaderamente similares se conecten. Veamos la explicación de los otros tipos de funciones de afinidad:

- **Gaussiana** $e^{-d^2/2\sigma^2}$ Penaliza fuertemente las distancias grandes para asegurar que sólo los píxeles verdaderamente similares se conecten.

- **Afinidad Lineal** $A(d) = \text{máx}\left(0, 1 - \frac{d}{\text{máx}(d)}\right)$
  Es la formulación más simple. Asigna una afinidad de 1.0 a la distancia nula y decrece de forma lineal y constante hasta llegar a 0.0 en la distancia máxima registrada en la matriz. En el procesamiento de imágenes complejas tiene un uso marginal, ya que no logra aislar correctamente los objetos, generando exceso de "ruido.ªl mantener conexiones débiles entre colores dispares.

- **Afinidad por Umbral** $A(d) = \begin{cases} 1 & \text{si } d < \epsilon \\ 0 & \text{si } d \geq \epsilon \end{cases}$
  Esta métrica genera un grafo de vecindad ϵ (*ϵ-neighborhood graph*). Emplea una lógica binaria estricta: si la distancia entre dos puntos no supera un parámetro de corte (ϵ), su conexión es absoluta; de lo contrario, se desconectan. Es particularmente útil en imágenes con formas geométricas puras, colores sólidos o de estilo vectorial, donde se busca una segmentación drástica sin tolerar gradientes.

- **Afinidad de Cauchy** $A(d) = \frac{1}{1+\left(\frac{d}{\sigma}\right)^2}$
  Basada en la distribución estadística de Cauchy, se caracteriza por poseer "colas pesadas". A diferencia de la Gaussiana, que decae exponencialmente a cero, la función de Cauchy mantiene valores residuales de conexión a largas distancias. Es la función óptima para la segmentación de **texturas y ruido** (como arena, asfalto o vegetación), ya que evita que el algoritmo fragmente de forma excesiva las pequeñas variaciones de los patrones rugosos.

- **Afinidad Multicuadrática Inversa** $A(d) = \frac{1}{\sqrt{d^2+\sigma^2}}$
  Esta función presenta una curva de decaimiento extremadamente suave debido a la presencia de la raíz cuadrada en el denominador. Es ideal para agrupar superficies con **gradientes muy sutiles y continuos** (por ejemplo, cielos con transiciones lumínicas o tonos de piel humana en fotografía), donde se requiere una alta tolerancia matemática a los cambios cromáticos progresivos.

- **Paso Fino (Gaussiana Truncada)** $A(d) = \begin{cases} e^{-\frac{d^2}{2\sigma^2}} & \text{si } d \leq 3\sigma \\ 0 & \text{si } d > 3\sigma \end{cases}$
  Su propósito es puramente de **eficiencia computacional**. La función Gaussiana estándar nunca alcanza el cero absoluto, llenando la matriz de conexiones con valores minúsculos que saturan la memoria RAM del sistema. El "paso fino" aplica un truncamiento, convirtiendo a cero absoluto cualquier conexión que supere las 3 desviaciones estándar (σ). Esto transforma el sistema en una **Matriz Dispersa** (*Sparse Matrix*), optimizando drásticamente el coste temporal y espacial del algoritmo sin comprometer la precisión topológica del resultado.

1.3. Celda 3: Preparación de la Matriz de Datos

```python
# 1. Cargar y redimensionar la imagen
archivo_imagen = 'frutas.png'
alto, ancho = 100, 100

img_original = io.imread(archivo_imagen)
img_small = transform.resize(img_original, (alto, ancho))

# 2. Extraccion de canales de color (RGB y HSV)
img_rgb = img_small[:, :, :3]
img_hsv = color.rgb2hsv(img_rgb)

R = img_rgb[:, :, 0].flatten()
G = img_rgb[:, :, 1].flatten()
B = img_rgb[:, :, 2].flatten()

H = img_hsv[:, :, 0].flatten()
S = img_hsv[:, :, 1].flatten()
V = img_hsv[:, :, 2].flatten()

# Truco Matematico: Coordenadas Polares para H y S
color_x = S * np.cos(2 * np.pi * H)
color_y = S * np.sin(2 * np.pi * H)

# 3. Caracteristicas del espacio (Posicion normalizada)
xx, yy = np.meshgrid(np.arange(ancho), np.arange(alto))
espacio_x = xx.flatten() / ancho
espacio_y = yy.flatten() / alto

# 4. Construccion de la Matriz Final (Pixeles x Caracteristicas)
alfa = 0.1  # Peso espacial
X_features_rgb = np.column_stack((R, G, B, alfa * espacio_x, alfa *
    espacio_y))
X_features_hsv = np.column_stack((color_x, color_y, V, alfa * espacio_x,
```

```
    alfa * espacio_y))
```

Listing 3: Extracción y normalización de características

En esta etapa del código se realiza una exhaustiva ingeniería de características, transformando la fotografía bidimensional en un conjunto de vectores matemáticos estructurados que el algoritmo de Clustering Espectral pueda procesar. El procedimiento se divide en las siguientes fases fundamentales:

- **Redimensionamiento:** La imagen original `frutas.png` se carga y se redimensiona obligatoriamente a una resolución de 100×100 píxeles. Este paso es una necesidad impuesta por la complejidad computacional del Clustering Espectral, la cual es de orden $\mathcal{O}(N^3)$. Una imagen de 100×100 genera 10000 píxeles. Si intentáramos procesar una imagen en alta resolución, generaríamos una matriz Laplaciana de proporciones inasumibles que desbordaría la memoria RAM de cualquier equipo computacional estándar.

- **Espacios de Color:** Antes de proceder a la transformación matemática, el código extrae la información cromática operando sobre dos espacios de color distintos: RGB y HSV. El análisis contrastado de estos dos modelos es fundamental para entender el rendimiento topológico del algoritmo bajo condiciones de iluminación no uniforme (sombras y brillos).

  - **El Espacio RGB (Rojo, Verde, Azul)** Es un modelo aditivo basado en un sistema de coordenadas cartesianas, donde cada color se representa como un vector dentro de un cubo tridimensional. Sus ejes corresponden a las intensidades de luz primaria (R, G, B). Aunque es el formato estándar nativo para la captura y visualización digital, presenta una grave limitación para tareas de segmentación geométrica: la alta correlación lumínica. Cuando una región del objeto se oscurece, las tres componentes vectoriales disminuyen su valor simultáneamente en dirección al origen nulo $[0, 0, 0]$. Esto dificulta que algoritmos basados en distancias puras separen matemáticamente un cambio de objeto (distinto color) de un simple cambio de volumen (distinta iluminación).
  - **El Espacio HSV (Matiz, Saturación, Valor):** Para mitigar las deficiencias topológicas del modelo RGB, la imagen se transforma mediante la librería `skimage.color` al espacio HSV. Este modelo emplea un sistema de coordenadas cilíndricas diseñado para aproximarse a la percepción visual humana, aislando la cromaticidad de la luminancia en tres componentes ortogonales: **H (Matiz):**, que define el color puro y se representa topológicamente como un ángulo (de 0° a 360°) sobre la circunferencia del cilindro cromático; **S (Saturación):**, indica la pureza o viveza del matiz frente a la escala de grises y geométricamente es la distancia radial desde el eje central del cilindro hasta la superficie; **V (Valor):** que representa la intensidad lumínica o el nivel de oscuridad absoluto (el eje vertical central del cilindro).

  La inmensa ventaja algorítmica de proyectar los datos al espacio HSV radica en la separación del canal lumínico (`V`). Al independizar la luz del color, permite que modelos matemáticos (como la métrica Coseno o la distancia sobre coordenadas polares) evalúen la similitud de dos píxeles basándose estrictamente en su firma cromática (`H` y `S`), otorgando al Clustering Espectral total inmunidad frente a la distorsión que generan las sombras arrojadas o los reflejos especulares.

- **Coordenadas polares:** el canal H es circular (0° es rojo y 360° también). Si se usa un valor lineal de 0 a 1 el algoritmo pensaría que el rojo del inicio y el del final son opuestos. Para evitarlo, usamos fórmulas trigonométricas $Scos(2\pi H)$ y $Ssin(2\pi H)$.

- **Coordenadas Espaciales Normalizadas:** Para conservar la coherencia topológica y que el algoritmo identifique qué píxeles son físicamente contiguos en la imagen original, se generan matrices de coordenadas espaciales (`espacio_x` y `espacio_y`) mediante la función `np.meshgrid`. Estos valores se normalizan dividiéndolos por el ancho y el alto totales. Esto asegura que las coordenadas físicas varíen en el intervalo $[0, 1]$, situándolas en la misma escala matemática que los canales de color y evitando que las distancias espaciales dominen artificialmente sobre las diferencias cromáticas.

- **Construcción de la Matriz Final:** Finalmente, mediante la función `np.column_stack`, las listas independientes se fusionan en las matrices definitivas (`X_features_rgb` y `X_features_hsv`). Cada píxel se define ahora como un vector en un espacio de 5 dimensiones. Por ejemplo, en el caso RGB, la morfología del vector es la siguiente:

  $$V = [R, G, B, (\alpha \cdot X), (\alpha \cdot Y)]$$

  El parámetro α (definido en el código como `alfa=0.1`) actúa como un hiperparámetro de peso espacial. Su función es crucial: permite calibrar la importancia de la proximidad física frente a la similitud cromática. Al ponderarlo con un factor de 0,1, el modelo prioriza fuertemente la segmentación por color, utilizando la posición física espacial únicamente como un factor de regularización local.

1.4. Celda 4: Cálculo de Distancias y Matriz de Afinidad

```python
# 1. Calculo de la Matriz de Distancias (Euclidea)
dist_matrix = cdist(X_features_hsv, X_features_hsv, metric='euclidean')

# 2. Matriz de Afinidad W
tipo_elegido = 'gaussiana'
parametro_elegido = 0.1

matriz_afinidad = calcular_mi_afinidad(dist_matrix, tipo=tipo_elegido,
    parametro=parametro_elegido)
```

Listing 4: Cálculo computacional del grafo de similitud

En este paso se construye el "grafo" calculando la separación entre cada uno de los 10000 píxeles.

- **Matriz de distancias (`cdist`):** compara cada uno de los 10000 píxeles con los otros 10000 usando una métrica. Genera una matriz de 10000×10000

- **Matriz de afinidad W:** le aplica la función de afinidad a esa matriz, conviertiéndola en "pesos de conexión"

El rendimiento del algoritmo de Clustering Espectral depende críticamente de la métrica espacial empleada para construir el grafo de similitud inicial. La función computacional permite emplear tanto las formulaciones matemáticas clásicas como métricas alternativas avanzadas para calcular la disimilitud entre vectores de características (píxeles). A continuación, se detallan las métricas más relevantes y su aplicabilidad en tareas de visión artificial.

1.4.1. Distancia Euclídea (Norma L_2)

$$d_{euclidea}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Mide la separación física en línea recta geométrica entre dos puntos en el espacio RGB o HSV. Es altamente **sensible a la magnitud del vector** (cantidad de luz o intensidad). Resulta ideal para separar colores diametralmente distintos bajo una iluminación uniforme, pero es muy propensa a la sobresegmentación y al fracaso morfológico ante fuertes gradientes de luz y sombras, ya que confunde la falta de iluminación con un cambio de objeto.

1.4.2. Distancia Coseno (Similitud Angular)

$$d_{coseno}(x, y) = 1 - \frac{\sum_{i=1}^{n} x_i y_i}{\sqrt{\sum_{i=1}^{n} x_i^2} \sqrt{\sum_{i=1}^{n} y_i^2}}$$

Evalúa el ángulo formado entre dos vectores, ignorando por completo su longitud o magnitud. Al ser **invariante a la escala de iluminación**, permite agrupar objetos basándose puramente en su cromaticidad (aislando la topología independientemente del sombreado). Sin embargo, presenta limitaciones topológicas con los colores acromáticos puros (blanco, negro, gris), ya que todos apuntan en la misma dirección (ángulo nulo), lo que puede generar anomalías en brillos especulares.

1.4.3. Distancia de Manhattan (Bloque de Ciudad o Norma L_1)

$$d_{manhattan}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

Calcula la suma de las diferencias absolutas a lo largo de cada dimensión (canales de color) formando ángulos rectos. Al no elevar las diferencias al cuadrado, esta métrica es **altamente robusta frente a valores atípicos extremos** (*outliers*), siendo ideal para imágenes afectadas por ruido impulsivo (sal y pimienta) o píxeles defectuosos, donde la Euclídea penalizaría de forma desproporcionada.

1.4.4. Distancia de Minkowski

$$d_{minkowski}(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

Constituye la generalización matemática de las métricas espaciales. Requiere un hiperparámetro p. Si $p = 1$, equivale a la distancia Manhattan; si $p = 2$, a la Euclídea. Valores de $p > 2$ exageran matemáticamente la influencia de la dimensión que presenta mayor varianza. Su uso permite una calibración fina de la sensibilidad del algoritmo ante la magnitud del color.

1.4.5. Distancia de Chebyshev (Norma Infinita)

$$d_{chebyshev}(x, y) = \max_i |x_i - y_i|$$

De todos los componentes del vector de características, esta métrica evalúa exclusivamente el canal que presenta la mayor diferencia absoluta, descartando el resto. En visión artificial industrial o control de calidad, resulta fundamental para clasificar un elemento como anómalo si **al menos un canal cromático** supera el umbral de tolerancia, ignorando si los demás coinciden con el patrón de referencia.

1.4.6. Distancia de Correlación

$$d_{correlacion}(x, y) = 1 - \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Comparte fundamento topológico con la métrica Coseno, pero aplica una centralización previa restando la media geométrica $(\bar{x}, \bar{y})$ a los vectores. Esta corrección la hace óptima para el análisis de imágenes biomédicas o datos con un sesgo base constante, ya que **aísla el patrón de variación intrínseco** independientemente del nivel de iluminación de fondo.

1.5. Celda 5: Cálculo del Número Óptimo de Clústeres (Eigengap)

```python
# 1. Matriz Dispersa
W_matriz = csr_matrix(matriz_afinidad)

# 2. Calculamos la Matriz Laplaciana Normalizada
L_norm = csgraph.laplacian(W_matriz, normed=True)

# 3. Calculamos los autovalores mas pequenos
max_k = 10
autovalores, _ = eigsh(L_norm, k=max_k, which='SA')
autovalores = np.sort(autovalores)

# 4. Buscamos el Eigengap para calcular el K optimo
gaps = np.diff(autovalores)
k_optimo = np.argmax(gaps[1:]) + 2

# Visualizacion del Eigengap
plt.figure(figsize=(8, 5))
plt.plot(range(1, max_k+1), autovalores, marker='o', linestyle='-',
    color='b')
plt.title('Heuristica del Eigengap: Autovalores del Laplaciano
    Normalizado')
plt.xlabel('Numero de componentes (k)')
plt.ylabel('Valor del Autovalor')
plt.xticks(range(1, max_k+1))
plt.axvline(x=k_optimo, color='r', linestyle='--', label=f'Corte optimo
    en k={k_optimo}')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
```

```
plt.show()
```

Listing 5: Cálculo matemático del Eigengap sobre el Laplaciano normalizado

Esta sección del código implementa la heurística del Eigengap, un método matemático que analiza la topología del grafo para deducir de forma autónoma el número óptimo de particiones (k). El proceso consta de cuatro fases algebraicas y una de visualización:

1. Se convierte la matriz de afinidad densa en una Matriz Dispersa Comprimida mediante la función `csr_matrix`. Una imagen de 100×100 píxeles genera una matriz de interconexiones de 10000×10000 (100 millones de elementos). Almacenar esta estructura en memoria estándar resulta ineficiente. El formato CSR comprime la matriz omitiendo los valores nulos, lo que reduce drásticamente el consumo de memoria RAM y acelera exponencialmente las operaciones algebraicas posteriores.

2. Se transforma la matriz de conexiones en una matriz de difusión utilizando `csgraph.laplacian`. Al incluir el parámetro `normed=True`, no se calcula el Laplaciano estándar, sino su versión normalizada simétrica:

$$L_{norm} = I - D^{-1/2} W D^{-1/2}$$

   Esta normalización es vital en el procesamiento de imágenes para evitar el problema topológico del "corte mínimo" aislado. Pondera las conexiones según el grado o nivel de conectividad de cada píxel, obligando al algoritmo a buscar agrupaciones que estén equilibradas en densidad y tamaño, y evitando así que aísle píxeles individuales ruidosos como si fueran clústeres enteros.

3. Se utiliza la función `eigsh` de la librería `SciPy` (potenciada por el motor ARPACK) para extraer exclusivamente los 10 primeros autovalores de la matriz Laplaciana. En lugar de calcular los 10000 autovalores posibles (lo cual colapsaría el sistema), se buscan únicamente los más pequeños desde el punto de vista algebraico (indicado por el parámetro `which='SA'`). Estos autovalores albergan la información fundamental sobre las particiones principales del grafo. Finalmente, `np.sort` los ordena de menor a mayor.

4. El código calcula matemáticamente el salto máximo entre autovalores consecutivos mediante `np.diff` para decidir el número de clústeres. La robustez de este paso reside en la instrucción `np.argmax(gaps[1:])`. Al utilizar el *slicing* en el vector (`[1:]`), se omite intencionadamente el primer salto. En un grafo idealmente particionado, el primer autovalor (λ_1) siempre es cero. Si el salto entre el primero y el segundo fuera inmenso y se evaluara toda la lista, el algoritmo sugeriría $k = 1$, lo que implicaría la ausencia total de segmentación. Al omitir el primer valor y sumar 2 al índice final (`+ 2`), se fuerza a la heurística a encontrar la partición óptima asumiendo la existencia mínima de una figura y un fondo ($k \geq 2$).

5. Por último se incluye la librería `matplotlib` para generar una gráfica de los autovalores, trazando una línea vertical punteada (`plt.axvline`) exactamente en la coordenada del k óptimo calculado. Esto sirve como prueba empírica y visual para demostrar que la elección de los clústeres no es un parámetro arbitrario escogido por el investigador, sino una deducción matemática extraída directamente de la estructura intrínseca de los datos.

1.6. Celda 6: Ejecución del Clustering Espectral y Visualización

```python
# 1. Configuracion del Modelo
sc = SpectralClustering(n_clusters=k_optimo, affinity='precomputed',
    random_state=42)

# 2. Entrenamiento y Asignacion de Etiquetas
etiquetas_planas = sc.fit_predict(matriz_afinidad)

# 3. Preparacion del lienzo para visualizacion
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 6))

axes[0].imshow(img_small)
axes[0].set_title("Imagen Original de Frutas")
axes[0].axis('off')

# Reconstruccion de la imagen segmentada
imagen_segmentada = etiquetas_planas.reshape(alto, ancho)
axes[1].imshow(imagen_segmentada, cmap='tab10')
axes[1].set_title("Segmentacion Espectral")
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

Listing 6: Entrenamiento del modelo y generación gráfica

En primer lugar, se crea el modelo de clustering utilizando la clase `SpectralClustering` de la librería de `scikit-learn`. El algoritmo cuenta con las siguientes fases:

1. Se construye el grafo mediante la matriz de afinidad. En nuestro caso, como `affinity = precomputed`, se toma la matriz ya calculada en la celda 4.

2. Una vez se tiene la matriz de afinidad W, se calcula la matriz de grados D y el Laplaciano $L = D - W$.

3. Aquí se ejecuta el motor del algoritmo. Mediante `eigen_solver='arpack'` se extraen los autovalores y autovectores de la matriz Laplaciana. Se toman los primeros k autovectores que forman una base de un nuevo sistema de coordenadas. El algoritmo proyecta cada píxel hacia este nuevo espacio. Los píxeles que estaban fuertemente conectados ahora son puntos en una bola muy densa, y los que no, están lejos.

4. Una vez que los datos han sido "desdoblados" y proyectados usando los autovectores, se lanza el algoritmo K-Means que funciona, pues los datos complejos se han transformada en grupos de puntos linealmente separables.

Este algoritmo cuenta con numerosos parámetros. Los principales son:

- `n_clusters`: define la dimensión del subespacio de proyección y el número de clusters a encontrar. Le pasamos la variable `k_optimo` calculado anteriormente.

- `affinity`: le indica al algoritmo como construir el grafo. En este caso `affinity='precomputed'` interpreta los datos de entrada como una matriz ya calculada. Otras opciones son: `'nearest_neighbors'` (usa fuerza bruta conectando los píxeles más cercanos con un 1), `'rbf'`(construye la matriz usando un núcleo Gaussiano, es decir, aplicando la fórmula $e^{-\frac{d^2}{2\sigma^2}}$), `precomputed_nearest_neighbors` (se

le pasa la matriz de datos calculada y construye el grafo mediante el vecino más próximo).

- `random_state`: aunque el cálculo de la matriz Laplaciana y de los autovalores son procesos exactos, la última parte del algoritmo utiliza internamente "K-Means" para asignar la etiqueta final a cada clúster. Este algoritmo funciona asignando los centroides al azar y ajustándolos poco a poco. Si estos puntos caen en sitios distintos cada vez que se ejecuta, el clúster podría variar. Poniendo `random_state=42`, nos aseguramos de que siempre se elijan los centroides al azar en las mismas posiciones. Al probar con otros valores enteros cambia la posición inicial de los centroides pero el resultado seguiría siendo similar.

- `eigen_solver`: define que algoritmo interno se va a utilizar para calcular los autovalores. Por defecto usa `'arpack'` de manera que en lugar de calcular todos los autovalores, aproximan la matriz extrayendo únicamente los k autovalores específicos que se necesitan.

- `assign_labels, n_init`: una vez que el algoritmo ha transformado los datos, hay que decidir como separar los grupos. Mediante el parámetro `assing_labels` se usa el algoritmo K-Means para clasificar los píxeles. También están las opciones `discretize`(menos sensible al azar) y `cluster_qr` (más rápida). Con `n_init = 10` se indica que se lanze el proceso 10 veces y muestre solo el mejor resultado.

- Como configuramos el modelo con `affinity='precomputed'` el algortimo ignora parámetros commo `gamma, degree, n_neighbors`. Hemos calculado previamente nuestra matriz de distancias y le hemos aplicado nuestra propia matriz de distancias.

Siguiendo con el código, mediante el método `fit_predict`, se toma la matriz, se extraen los clústeres y se devuelven las etiquetas finales para cada elemento. En nuestro ejemplo, devolvería un vector de 10000 componentes de $0, 1, 2, 3$.

En el resto de la celda, se prepara el espacio gráfico donde vamos a visualizar el resultado. Como `fit_predict` devolvió un vector de 10000 componentes, mediante `reshape` volvemos a convertirla en una imagen de 100×100 píxeles.

Finalmente, le damos un mapa de colores `cmap='tab10'` para que asigne un color llamativo a cada etiqueta distinta y podamos ver la segmentación a simple vista.

1.7. Conclusiones

En este proyecto hemos puesto a prueba la capacidad del algoritmo de Clustering Espectral aplicándolo a un escenario complejo del mundo real: la segmentación no supervisada de una fotografía de frutas. Tras diseñar e implementar el modelo, con los resultados obtenidos hemos podido extraer conclusiones muy reveladoras sobre cómo procesan la información visual este tipo de algoritmos.

- En primer lugar, hemos comprobado que analizar únicamente el color de los píxeles no es suficiente para segmentar imágenes naturales. El verdadero éxito de nuestro modelo ha residido en la fase previa de preparación de los datos.

- En los algoritmos tradicionales de agrupamiento, el desarrollador suele tener que adivinar o introducir a mano el número de grupos (k) que espera encontrar. Sin embargo, al transformar nuestras interconexiones en una Matriz Laplaciana y analizar

el comportamiento de sus autovalores, el sistema fue capaz de deducir matemáticamente en cuántas partes naturales se dividía la escena. Esto elimina por completo el sesgo humano y demuestra que el modelo puede adaptarse de forma autónoma a la imagen que se le presente.

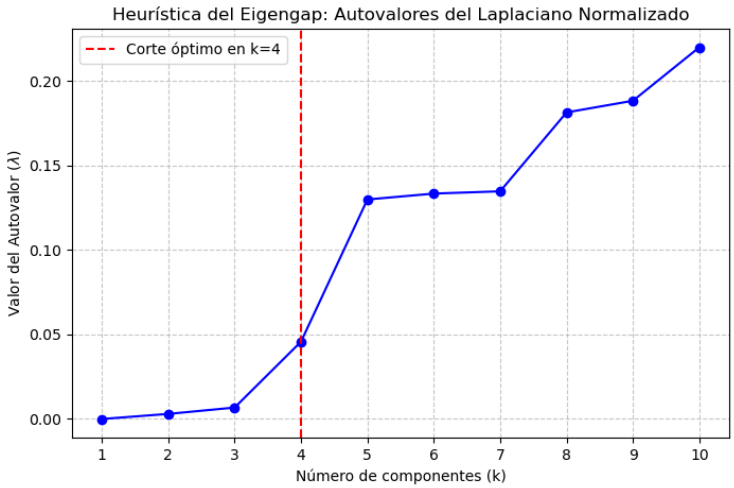

Figura 1: Heurística del Eigengap: El algoritmo deduce por sí mismo el número óptimo de clústeres al analizar los saltos en los autovalores.

- Al trasladar nuestros datos al subespacio matemático de los autovectores, el algoritmo consiguió "desenrollar" las formas complejas que tienen las imágenes. De esta manera, cuando aplicamos el paso final de clasificación, el modelo pudo separar las frutas del fondo con gran facilidad. Tal y como evidencia el resultado final (Figura 2), el Clustering Espectral ha logrado aislar la forma real de las frutas ignorando casi por completo las distorsiones causadas por la iluminación irregular.

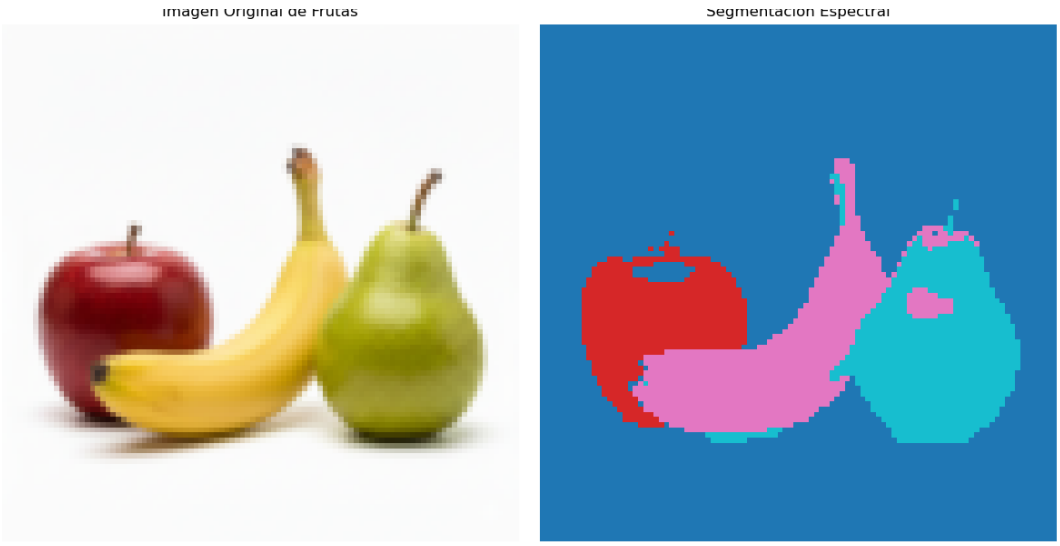

Figura 2: Comparativa visual: Fotografía original (izquierda) frente al resultado de la segmentación espectral (derecha), mostrando una delimitación muy precisa de los objetos.